Dr. ML - Multi-Disease Predictor

Project Walkthrough

This project is a complete machine learning application for predicting multiple diseases using a FastAPI backend and a Streamlit frontend. It combines disease-specific training pipelines, reusable preprocessing utilities, and a user-facing clinical prediction interface.

1. Project Goal

The system helps healthcare users estimate the probability of two conditions: diabetes and heart disease. The application captures patient features, runs them through trained ML models, and returns a prediction with a probability score.

2. Core Features

- Diabetes risk prediction using patient health metrics.
- Heart disease risk prediction using clinical indicators.
- FastAPI backend service for secure prediction requests.
- Streamlit frontend for interactive input and instant results.
- Reusable model training scripts and hyperparameter configuration.
- Downloadable PDF project walkthrough for developers and stakeholders.

3. Application Structure

The repository is organized into frontend, backend, training, and shared utility modules. This separation makes the app easier to scale, test, and maintain.

• src/frontend: Streamlit user interfaces for diabetes and heart disease prediction. • src/Backend: FastAPI application, routing layer, and prediction service logic. • src/training: model training pipelines for each disease and configuration files. • src/Common: shared preprocessing utilities used during training and prediction. • model_dir: trained joblib model files. • docs: generated project documentation and exportable walkthrough PDFs.

4. Data Flow

The user enters clinical values in the Streamlit form. The frontend sends a JSON payload to the FastAPI endpoint /api/predict. The backend identifies the disease type, loads the corresponding trained model, and returns the prediction result with a probability value.

5. Training Workflow

Each disease model is trained from a CSV dataset with risk attributes and labels. The training pipeline includes preprocessing steps such as zero-to-NaN replacement, median imputation, scaling, and model fitting. Hyperparameters are loaded from YAML files to support repeatable experimentation.

6. Technologies Used

- Python 3 for application logic and model training.
- Streamlit for interactive web-based prediction interfaces.
- FastAPI for backend API endpoints and request handling.

- Pandas, NumPy, and scikit-learn for data processing and ML workflows.
- Joblib to persist trained pipeline objects.
- ReportLab to export documents such as this walkthrough PDF.

7. How to Run the Project

1. Install dependencies from requirements.txt. 2. Configure environment variables in .env using the template in env_template.txt. 3. Start the API: uvicorn src.Backend.main:app --reload 4. Launch the frontend: streamlit run src/frontend/app.py 5. Open the app in the browser and submit patient details for prediction.

8. Prediction API

The API exposes a POST endpoint at /api/predict. It accepts the disease name and a feature dictionary, then responds with the predicted class and probability. This design keeps the model logic separate from the UI while making integration easy.

9. Notes for Maintainers

The project is designed for extension. New disease predictors can be added by creating a new training script, model artifact, and frontend page while reusing the same API pattern. This architecture supports future healthcare use cases with minimal structural changes.

Prepared for project onboarding, maintenance, and stakeholder review.